

From one task to a self-running system

Automations, plugins and dynamic workflows on your payments service

Today: core first, then extras

One repo all the way through: payments-api-demo, its legacy jobs/, and its Azure DevOps board.

CORE · LIVE · 60 MIN

The system

- | 01 **Devin suggests the automations.**
The opener: Devin reads your org's sessions and proposes what to automate.
- | 02 **Automations on Azure DevOps.**
Work item assigned → session → PR → board updated. Conditions, limits, queues, human gates. Self-healing CI.
- | 03 **Plugins.**
Rules, skills, hooks and the ADO MCP as one versioned unit. Knowledge and playbooks, upgraded.
- | 04 **Dynamic workflows.**
Migrate five legacy cron jobs in parallel: split, fix, verify, roll up. Interrupt and resume.

EXTRAS · ONE SLIDE EACH · DEMO IF TIME

Around the system

- | 01 **CLI ⇄ Cloud.**
Start on the laptop, hand off to the cloud, SSH into the VM, pick it back up.
- | 02 **Code scans.**
Security, dead code and test gaps on a schedule; assign findings to Devin.
- | 03 **Devin On-Call · beta.**
Devin as first responder: triage, incident notes, fix PR, post-mortem.
- | 04 **Devin Migrations · beta.**
The migration workflow as a product: inventory, plan, parallel execution, roll-up.

THE THREAD

Every demo ends in a PR you could merge and a board that updated itself.

Opener: Devin suggests the automations

You do not need to know what to automate. Devin has watched the last month of sessions and knows where the repetition is.

AUTOMATIONS · CREATE · SUGGEST FOR ME

What happens on screen

Automations → Create → “Suggest for me”



Devin scans recent sessions, repos and integrations



Proposals with trigger + prompt pre-filled



Pick “implement assigned ADO work items” → editor opens

WHAT TO POINT AT

The editor, top to bottom

- | Trigger: webhook from an ADO Service Hook (work item updated).
- | Conditions: regex on the payload. Only items tagged “devin”, only when Assigned To changed.
- | Prompt: calls the plugin skill /payments-workshop:ado-work-item. Short, because the skill carries the detail.
- | Limits: ACUs per run, runs per hour, concurrency group + queueing, network policy, security profile.
- | “Generate with Devin”: the fallback that builds the automation with you in a session.

MOVE ON WHEN

The suggested automation is saved and disabled. We enable it live in the next block.

Core A: automations on Azure DevOps

Their board, their tickets, their CI. Devin is a team member the board can assign to.

A1 · WORK ITEM → PR

One ticket, hands off until review

Refunds work item assigned to Devin in ADO



Service hook fires → conditions match → session starts



Skill: read item, Active, branch, implement, verify



PR “AB#42: Refunds...” → comment on item → Resolved

A2 – A4 · THE CHAIN AROUND IT

Automations that watch each other

| 01 Self-healing CI.

Check run fails → fix on the same branch → Devin Review comments → merge conflicts auto-fixed. A no-loop condition guards it.

| 02 Nightly chore on a schedule.

Dependency bumps, README drift, mechanical lint. No PR if nothing to do. “Run now” to show it.

| 03 Incident webhook.

Alert → reproduce → RCA to Teams → fix PR → ADO bug linked. The hand-built version of On-Call.

| 04 Human gate.

Ambiguous acceptance criteria? Devin asks in Teams and waits. Nothing is closed by Devin; Resolved is the ceiling.

THE RULE

Nothing goes on a trigger you have not watched succeed by hand. Limits and queues make it safe to leave on.

Core B: plugins

Everything you taught Devin about this repo, versioned, installable and managed by Devin itself.

BEFORE · KNOWLEDGE, PLAYBOOKS, SKILLS

Three places, loosely coupled

- | 01 **Knowledge notes.**
Flat text, org-wide, hard to scope. Now migrated for you into a “knowledge” plugin as skills.
- | 02 **Playbooks.**
Reusable prompts. Good for one flow, no tools or rules attached.
- | 03 **Skills in the repo.**
Files Devin can invoke. Great, but each repo reinvents the house rules.

NOW · ONE PLUGIN: payments-workshop

One unit, one version, one install

- | 01 **AGENTS.md + rules/.**
Always-on house rules and an ADO rule that triggers on AB#. Decimal money, UTC, tests per endpoint.
- | 02 **skills/.**
run-and-verify, ado-work-item, legacy-job-migration. Slash-invokable, used by the automations.
- | 03 **MCP + hooks.**
Azure DevOps MCP ships with the plugin. Installed per person, org or enterprise; required or available-to-install.
- | 04 **Devin manages it.**
“Add a rule: never log account numbers” → Devin edits the plugin, asks approval, rolls it out.

DEMO BEAT

Show Customize → Plugins with the migrated knowledge plugin next to payments-workshop, then let Devin add a rule live.

Core C: dynamic workflows

Five legacy cron scripts in jobs/. One target: app.scheduler. One prompt, twelve sessions, one PR.

THE SCRIPT · .devin/skills/migrate-jobs

Orchestration as code

“Migrate every job in jobs/, one agent per job, verify each, roll up”

1 INVENTORY

Lite agent reads jobs/README and lists the five units

2 MIGRATE

reconcile · branch

export · branch

fx_rates · branch

... · branch

3 VERIFY

test + review

test + review

test + review

test + review

4 ROLL UP

Merge passing branches, one PR, “needs a human” list

pipeline(): a job moves to verify as soon as its migration lands. Structured output between layers. Each agent is a normal session with its own ACU cap.

WHEN IT EARNS ITS KEEP

Five or more items and more than one step. Below that, an automation is enough.

WHAT SURPRISES PEOPLE

The demo beats

| 01

Interrupt one agent mid-flight.

“Use JSON, not pickle” → it adapts; the other four never notice.

| 02

Kill the run, resume by run id.

Finished agents replay from the journal instantly. Only unfinished work re-runs.

| 03

Verification is a different agent.

The migrator never grades its own work.

| 04

It becomes a skill.

Once it worked, the script is saved; next time it is “run the migrate-jobs workflow”.

Extra 1: CLI ⇌ Cloud

The same Devin on your laptop and in the cloud. Work moves, context comes with it.

WHAT IT IS

One agent, two places

- | devin in any repo: local sessions, Fusion mode, subagents, sandboxed shell.
- | `/handoff` sends the session to the cloud with its full context; `/pickup` brings it back.
- | `devin ssh <session>` drops you into the cloud VM; `devin forward <session> 8000` brings its ports to localhost.
- | Also hands off from Claude Code and Cursor. Same plugins install locally: `devin plugins install`.

WHY IT MATTERS

Engineers stop choosing between “my machine” and “Devin’s machine”. Long tasks stop dying with the lid.

THE DEMO · 5 MIN

Round trip on one ticket

- | 01 **Start the FX-filter ticket locally.**
devin in payments-api-demo, paste the work item.
- | 02 **`/handoff`.**
Meeting time: the cloud session carries on. Show the worklog picking up mid-thought.
- | 03 **devin ssh, then devin forward 8000.**
curl /ops/jobs inside the VM; open Swagger on your laptop from the cloud VM.
- | 04 **`/pickup`.**
Back on the laptop with the branch and the conversation.

Extra 2: code scans

Whole-repo scans with a findings tab, on a schedule, with a button that makes Devin fix it.

WHAT IT IS

Findings you can assign

- | Scan types: security, performance, dead code, test coverage, accessibility, or a custom brief.
- | Findings tab per scan: severity, file, explanation, status API. Re-scan only new commits.
- | Assign to Devin: a session per finding, or bulk remediate; fix PRs come back auto-reviewed.
- | Scheduled scans + multi-repo. Security Swarm adds threat models and chained attack paths.

WHY IT MATTERS

Security debt becomes a queue Devin works through, not a quarterly audit.

THE DEMO · 5 MIN

Scan the demo repo live

- | 01 **New scan: security + dead code + coverage.**
The legacy jobs/ folder is full of on-purpose findings.
- | 02 **Read three findings.**
pickle cache in refresh_fx_rates, os.system in reconcile_ledger, untested GET /transactions/{id}.
- | 03 **Assign to Devin.**
One finding → fix PR. Show the review comment and the finding flipping to resolved.
- | 04 **Put it on a schedule.**
Weekly, scan new commits only. Tie back to Core A: an automation on the scan result.

Extra 3: Devin On-Call

BETA · EARLY ACCESS

Devin as the first responder. Same outcome as the incident automation you saw, zero prompt engineering.

WHAT IT IS

A responder, not a prompt

- | Triggered by a PagerDuty incident or a Slack / Teams on-call group mention.
- | Triage: reproduces, reads logs and recent PRs, posts incident notes as it goes.
- | Opens a fix PR and pings the human on call; never merges or closes.
- | On resolve: drafts the post-mortem with timeline and follow-ups.

THE DEMO · 5 MIN

Page Devin

- | 01 **Fire the alert.**
POST /ops/incidents/fx_timeout behind the monitor, or a PagerDuty test incident.
- | 02 **Watch the notes appear.**
Symptom, blast radius, suspected cause, in the incident itself.
- | 03 **Fix PR lands.**
Same bug as the ADO item “FX provider timeout returns 500”: 503 + Retry-After.
- | 04 **Resolve → post-mortem draft.**
Contrast with automation 04 on the earlier slide: this is that, productised.

WHY IT MATTERS

MTTR drops before anyone opens a laptop. You are seeing this before general availability.

Extra 4: Devin Migrations

BETA · EARLY ACCESS

The dynamic workflow from Core C, as a product: inventory, plan, parallel execution, roll-up.

WHAT IT IS

Migrations as a managed run

- | Point it at a codebase and a target: framework, language version, pattern, or here, jobs/ → app.scheduler.
- | Devin builds the inventory and a plan you review before anything runs.
- | Units execute in parallel with per-unit verification and progress you can watch.
- | Roll-up: what merged, what needs a human, PRs ready for review.

THE DEMO · 5 MIN

Same jobs/, no script

- | 01 **Open Migrations, pick payments-api-demo.**
Target: “legacy cron scripts onto app.scheduler”.
- | 02 **Review the plan.**
Five units, dependencies, estimated cost. Approve.
- | 03 **Watch it run.**
Progress per unit; needs-attention list as it forms.
- | 04 **Compare with Core C.**
Workflow: you wrote the orchestration. Migrations: the orchestration is the product.

WHY IT MATTERS

Multi-week migrations become a run you supervise. Beta today; exact flow shown from the beta UI.

Take it home

Everything shown is in payments-api-demo: the plugin, the workflow, the automations and the walkthrough.

THIS WEEK

Three things to switch on

- | 01 **Run “Suggest for me” on your org.**
Pick one suggestion, add conditions and limits, leave it disabled, watch it by hand once.
- | 02 **Turn one repo’s knowledge into a plugin.**
The knowledge plugin is already there; add your rules and the ADO MCP.
- | 03 **Find your first fan-out.**
Five or more similar items, more than one step. Start with a slice of two.

IN THE REPO

Where things live

- | 01 **demo/WORKSHOP.md**
Demo order, prompts, seeding, fallbacks.
- | 02 **demo/automations/**
The four automations and the opener, ready to paste.
- | 03 **.devin/plugins/payments-workshop**
Rules, skills, ADO MCP. Install at org scope.
- | 04 **.devin/skills/migrate-jobs**
The dynamic workflow script and its skill.

THE MEASURE

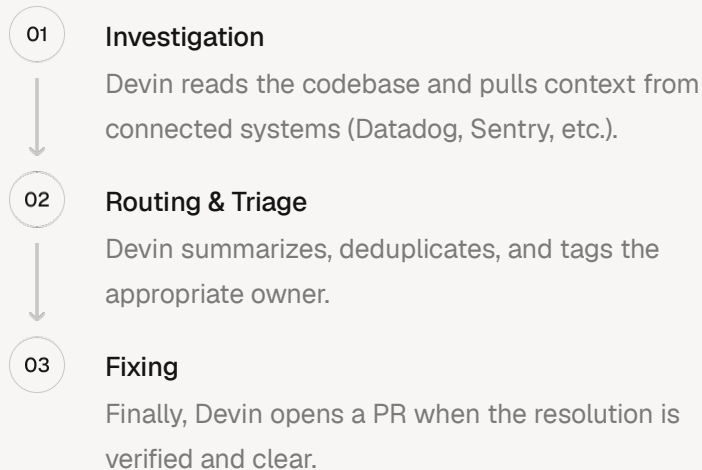
Fewer human hours per ticket, not more sessions.

Automation Triggers

Input Sources

- **Custom webhooks.** Any external system that POSTs an event.
- **Schedules.** Daily, weekly, or custom intervals (replaces current scheduled-sessions flow).
- **Github Events.** PR opened, issue activity, or CI results.
- **Slack Events.** Message posted in a channel or alert firing.
- **Linear Events.** Tickets opened, state changes, etc.

Autonomous Execution Loop



Event-Driven Use Cases with Devin Automations

Vulnerability Remediation

Automatically scan every PR for security risks.
Have Devin scope + apply patches safely.



Incident Management

Automate RCA and reduce MTTR by paging
Devin instead of your teammates.



Code Review

Automatically scan every PR for security risks.
Have Devin scope + apply patches safely.



Unit Test Coverage

Check every PR for new, untested code. Have
Devin automatically write + apply new tests.



Project Management

Assign a ticket to Devin for automatic scoping,
planning, and execution.



Communication Channels

Just @Devin on Slack to assign a task. Have
team members in-channel for visibility.

